Программная инженерия

Лекция 8: Архитектура программных систем: проектирование и эволюция

Денисов Артём Владимирович

Уральский Федеральный Университет

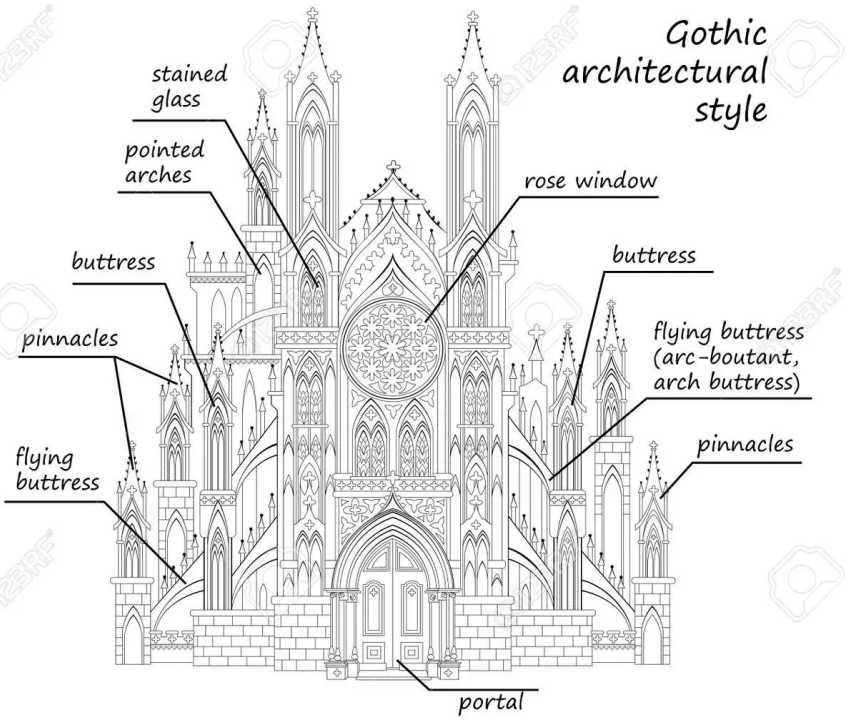
2026

Что такое архитектурный стиль?

Пример архитектурного стиля

Пример: готический стиль

- крестовый свод
- две арки вместо четырёх романских
- высокие расчленённые стены
- стрельчатые арки



Экземпляр архитектурного стиля

Пример: готический стиль

- крестовый свод
- две арки вместо четырёх романских
- высокие расчленённые стены
- стрельчатые арки

Плохая архитектура?

Нет архитектуры, лучшей во всём

«Хорошая архитектура всегда зависит от контекста»

Почему?

- разные цели зданий
- разные бюджеты и материалы
- разный климат и среда
- разные требования к эксплуатации

Что мы называем архитектурой?

Определение

- система ключевых решений об устройстве объекта
- состав частей, связи и границы
- форма, подчинённая функции и ограничениям

Следствие

- архитектура отвечает не только на вопрос «как выглядит?»
- но и на вопрос «как устроено и работает?»

Архитектура — это работа с ограничениями

В строительстве

- назначение здания
- безопасность
- стоимость
- долговечность

В инженерии вообще

- нельзя оптимизировать всё сразу
- каждое решение что-то усиливает
- и что-то делает дороже

Архитектура ПО и дизайн ПО

Ключевое различие

- **Архитектура ПО** определяет, *как устроена система в целом*.
- **Дизайн ПО** определяет, *как устроены отдельные части системы*.

Коротко

- Архитектура — уровень системы.
- Дизайн — уровень компонентов, модулей и кода.

Что делает решение архитектурным

Архитектурным решение становится, если оно

- определяет **структуру системы**: части, границы, взаимодействия;
- влияет на **качества системы**: масштабируемость, надёжность, сопровождаемость, безопасность;
- затрагивает **несколько подсистем** или систему в целом;
- является **дорогим в изменении** после реализации;
- задаёт рамки для множества последующих дизайн-решений.

Главная функция архитектуры

Архитектура — это управление стоимостью изменений

- сделать изменения локальными
- снизить риск каскадных последствий
- сохранить систему развиваемой

Примеры архитектурных решений

- отделить бинарные файлы от транзакционных данных
- выбрать асинхронную обработку
- провести границы между доменами

Архитектура $\neq$ технологии

Ошибка

- «наша архитектура — Kubernetes, Kafka, PostgreSQL»

На самом деле архитектура — это

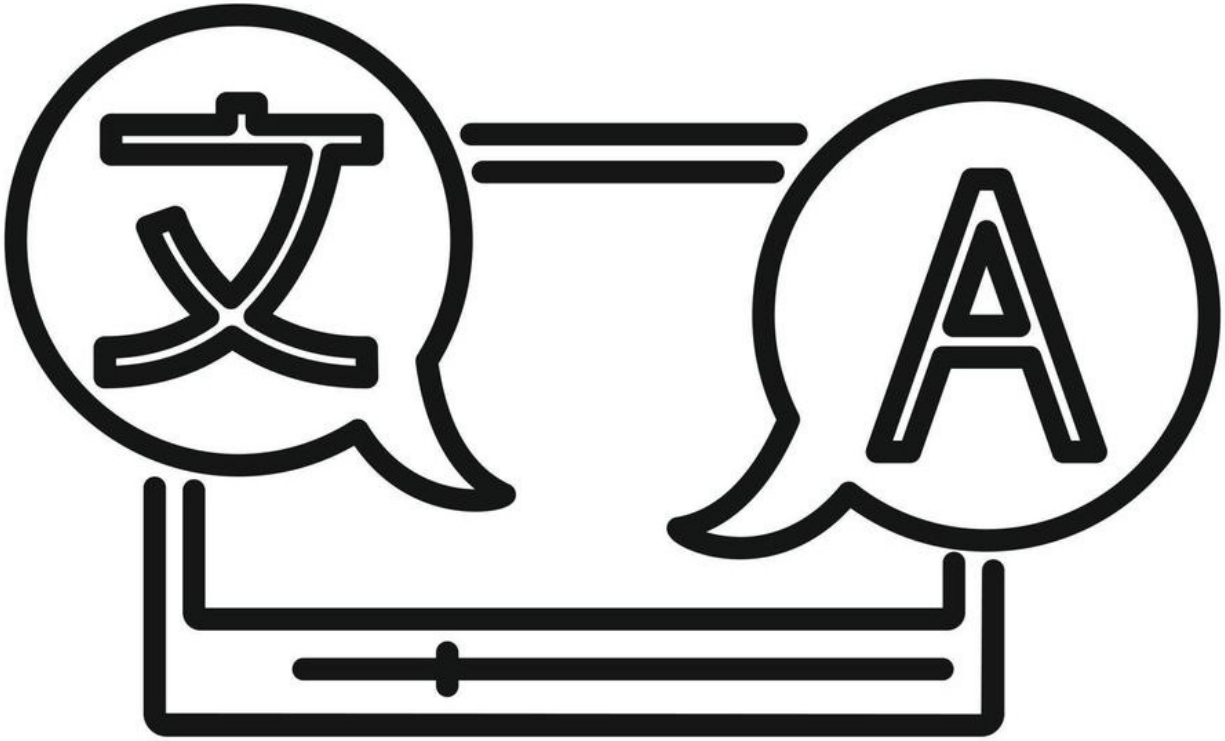
- как разделена ответственность
- какие части независимы

Coupling и Cohesion

Подробное объяснение

- https://enterprisecraftsmanship.com/posts/cohesion-coupling-difference/
- RU: https://habr.com/ru/articles/568216/

Пример. Сервис автоматического перевода видео



Требования заказчика: сервис перевода видео

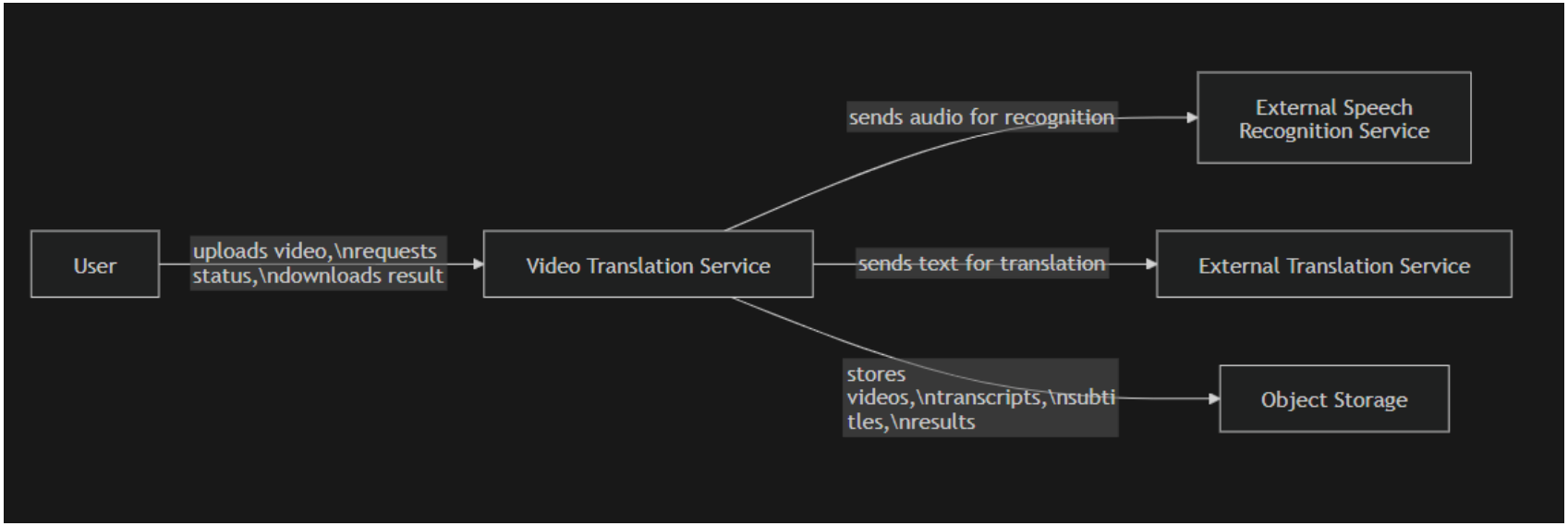
Что должен уметь сервис

- загрузка видеофайла пользователем
- выбор исходного и целевого языка
- автоматическое создание перевода и субтитров
- выдача готового результата для скачивания

Ожидания к системе

- обработка длительных задач в фоне
- понятный статус выполнения
- поддержка нескольких языков
- приемлемое время и стоимость обработки

Контекст системы



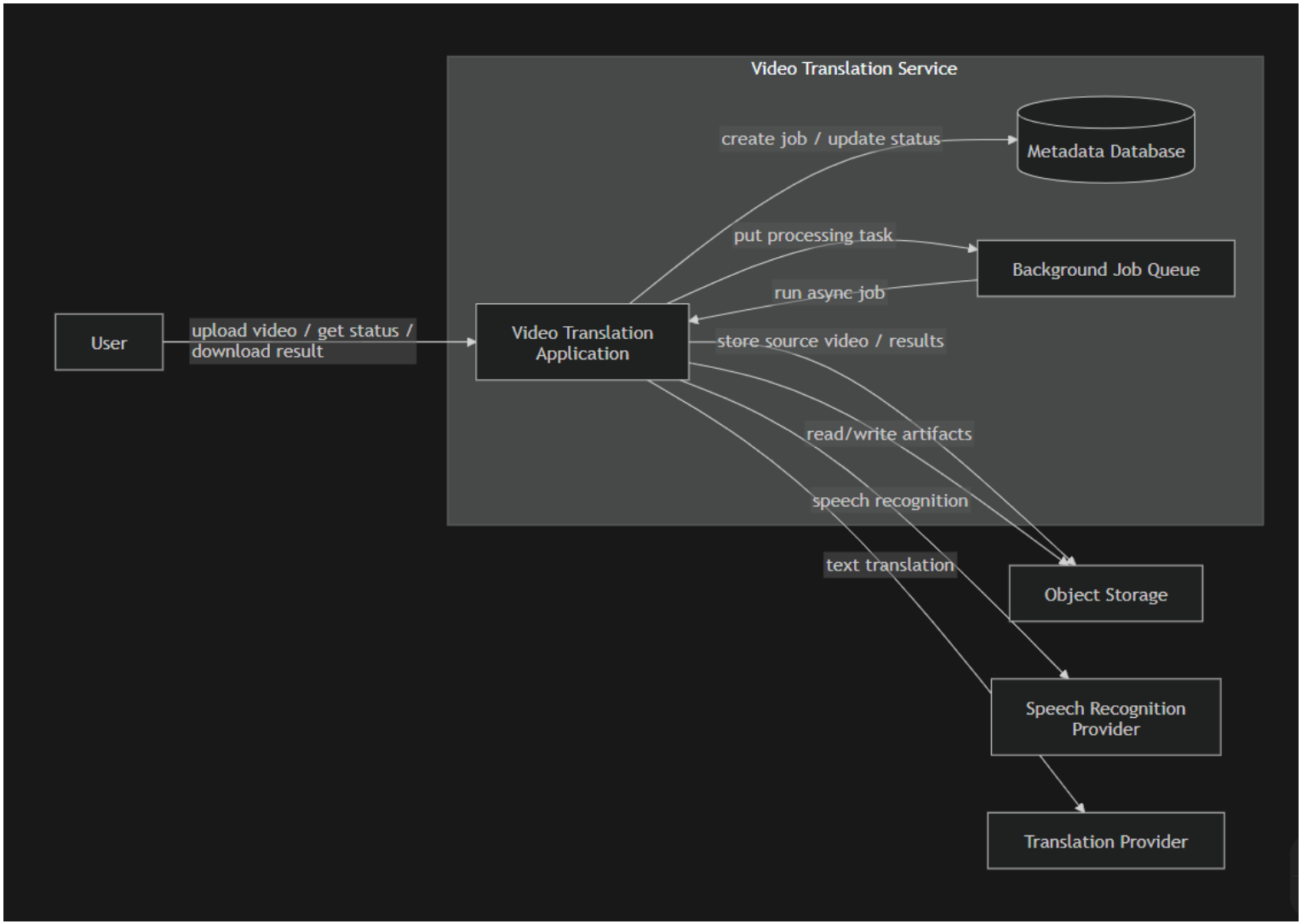
Абстрактное представление

Этап 1. Архитектурные решения

Архитектурное решение

- Не пытаться обрабатывать видео полностью в рамках одного HTTP-запроса
- Вместо этого:
  - система создаёт job/задачу;
  - пользователь проверяет статус или получает уведомление по завершении.

Этап 1. Начальный контур



Начальный контур

Монолитная архитектура

Что это такое

- Монолит — это приложение с единой кодовой базой
- Основная логика системы работает как одно целое
- Обычно такая система собирается и развёртывается как единый блок
- Внутри могут быть модули и слои, но снаружи это одна система

Когда это уместно

- старт продукта
- быстрое развитие требований
- приоритет — простота и скорость поставки

Плюсы монолитной архитектуры

Сильные стороны

- Проще начать разработку
- Проще локально запускать и отлаживать систему
- Проще собирать, тестировать и выпускать первую версию
- Меньше инфраструктурной и DevOps-сложности

Минусы монолитной архитектуры

Ограничения

- С ростом системы увеличивается связность между частями
- Даже маленькое изменение может требовать релиза всей системы
- Трудно масштабировать только одну нагруженную часть
- Сложнее изолировать сбои и зоны ответственности команд
- Большой монолит становится дорогим в изменении и сопровождении

Этап 1. Пилот: что фиксируется

Архитектурная основа

- Один сценарий: видео $\rightarrow$ перевод $\rightarrow$ субтитры.
- Обработка строится вокруг асинхронного job.
- Метаданные и файлы разводятся по разным хранилищам.
- Артефакты pipeline становятся явными сущностями.

Следующий шаг

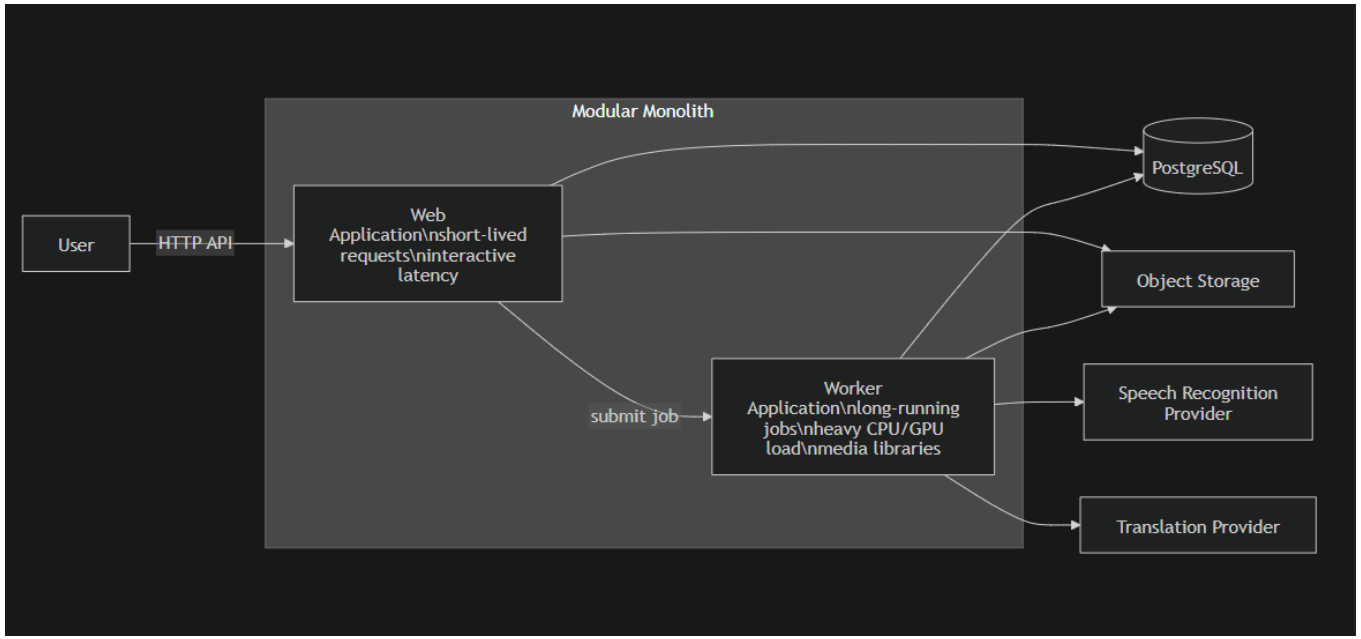
- Следующий шаг — собрать это в рабочий монолит.

Этап 2. Архитектурные решения

Архитектурное решение

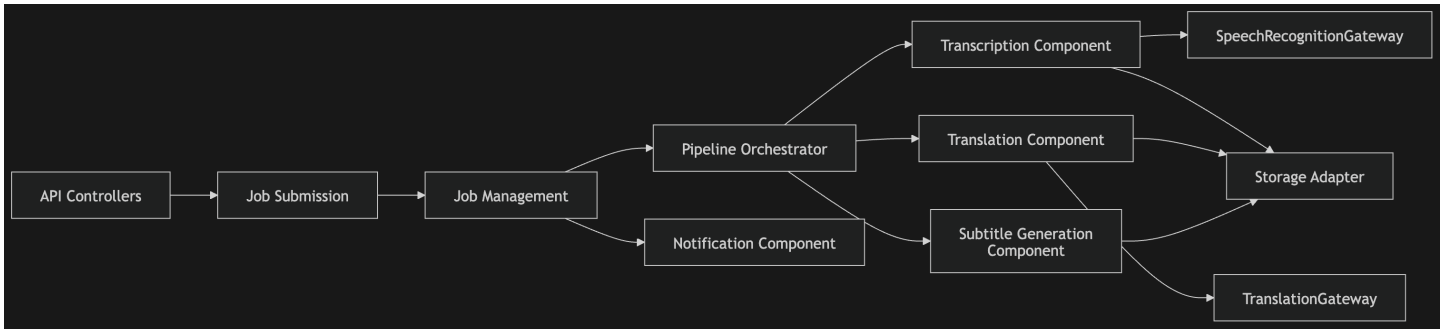
- Реализовать первую версию как модульный монолит
- Разделить систему на web-роль и worker-роль в одной codebase
- Организовать монолит по слоям: API, application, infrastructure
- Сделать Job Management единым владельцем жизненного цикла задачи

Этап 2. Модульный монолит



Монолит с разделением на web и worker роли

Внутри монолита



Компоненты монолита: управление задачами и pipeline

Первая рабочая архитектура

Что фиксируется

- Одна codebase и единый релиз
- Web и Worker — разные роли одного приложения
- Внутри вводятся слои и явные модули
- Job Management централизует жизненный цикл задачи

Что это даёт

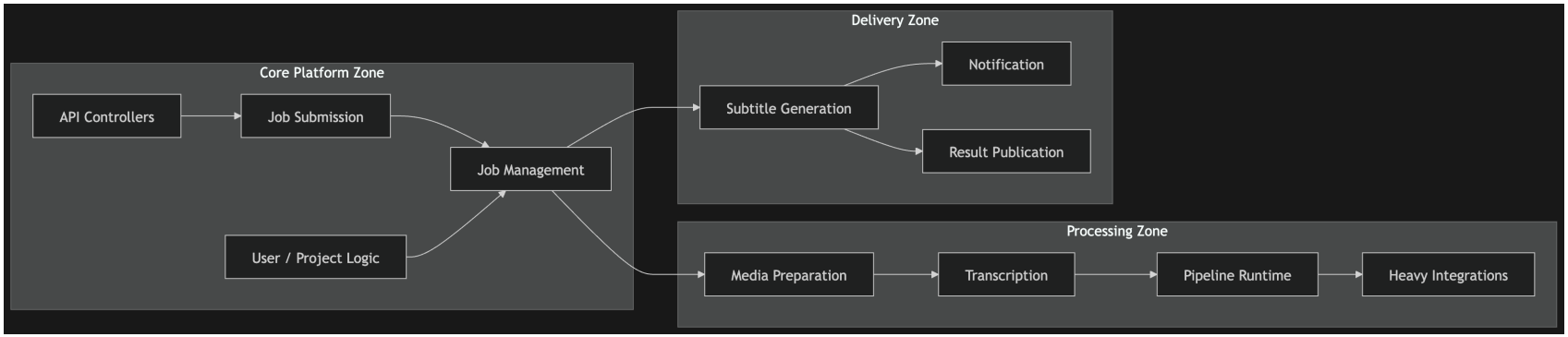
- Система быстро развивается и рефакторится

Этап 3. Архитектурные решения

Архитектурное решение

- Зафиксировать внутри монолита разные контуры нагрузки и ответственности

Этап 3. Монолит с меньшим coupling



Внутри появляются будущие границы разделения

Где возникает напряжение

Новые сигналы

- Web и processing живут под разной нагрузкой
- Media-обработка требует тяжёлого окружения
- Один релиз задевает несвязанные изменения

Этап 4. Архитектурные решения

Архитектурное решение

- Выделить Processing Service из монолита в отдельный сервис
- Оставить в Core Platform пользователей, API и управление job
- Ввести явный контракт между Core и Processing
- Развести владение данными и отказаться от общей базы

Микросервисная архитектура

Что это такое

- Микросервисы — это набор небольших автономных сервисов
- Каждый сервис отвечает за одну бизнес-способность
- Сервисы взаимодействуют через явные API или события
- Каждый сервис развивается и развёртывается независимо

Когда это уместно

- большая и сложная система
- разные профили нагрузки
- несколько команд
- потребность в независимых релизах

Плюсы микросервисной архитектуры

Сильные стороны

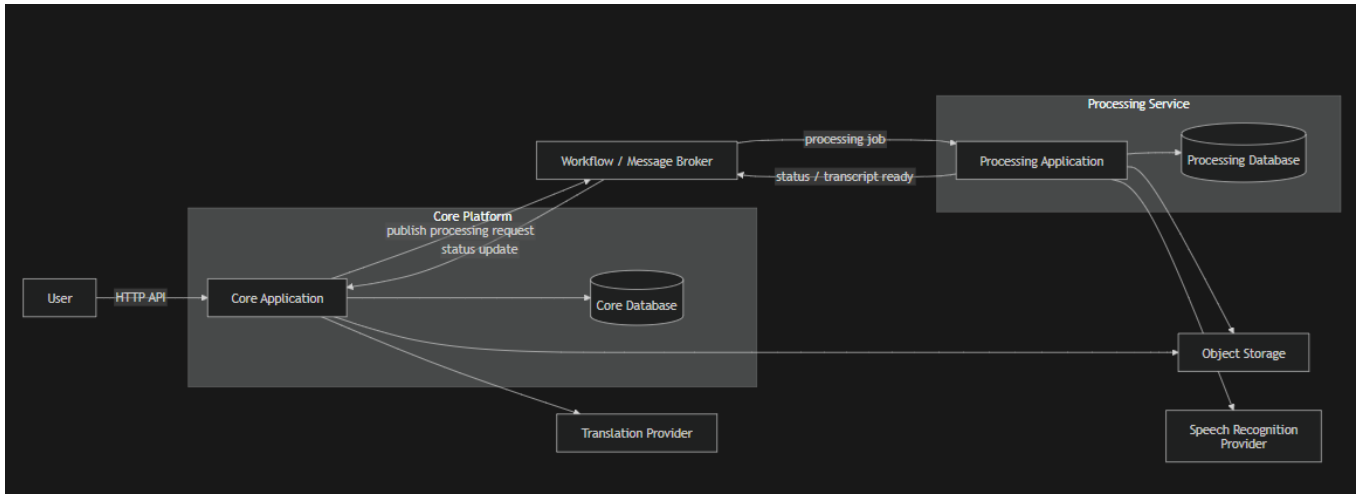
- Независимое развертывание сервисов
- Независимое масштабирование разных частей системы
- Более чёткие границы ответственности

Минусы микросервисной архитектуры

Ограничения

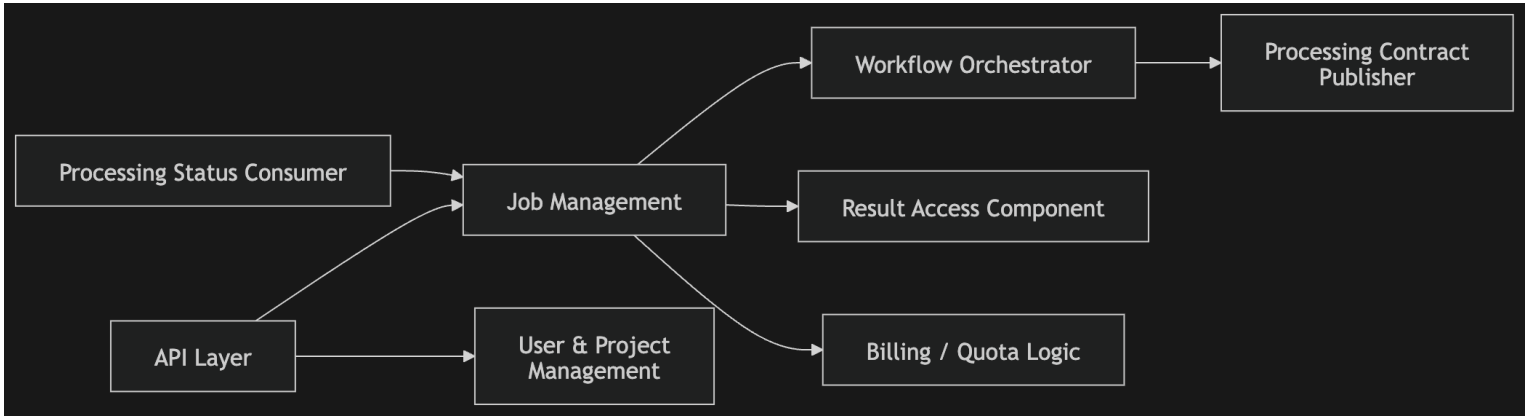
- Растёт сложность взаимодействия между сервисами
- Появляются сетевые сбои, таймауты
- Усложняется согласованность данных
- Требуются зрелые DevOps, мониторинг и автоматизация
- Отладка распределённой системы заметно сложнее

Этап 4. Отделение микросервиса

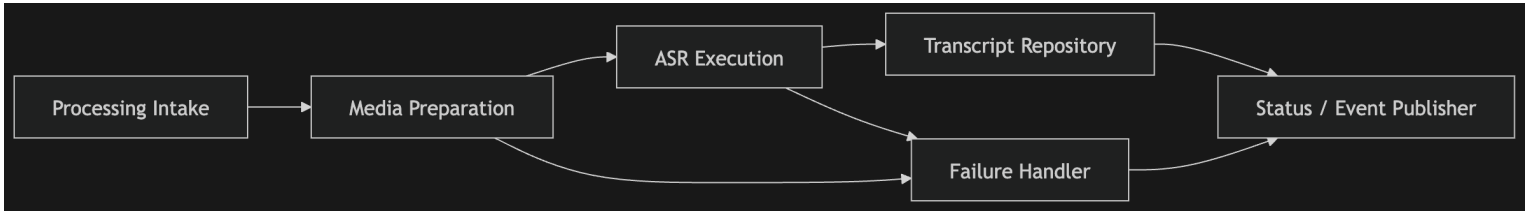


Разделение на Core Platform и Processing Service

Core и Processing



Компоненты Core Platform



Компоненты Processing Service

Выделение Processing

Что меняется

- Core сохраняет API, пользователей и job
- Processing забирает media pipeline и ASR
- Между ними вводится явный контракт
- Общая база данных исчезает

Архитектурный эффект

- Тяжёлая обработка масштабируется отдельно

Цена новой границы

Что теперь обязательно

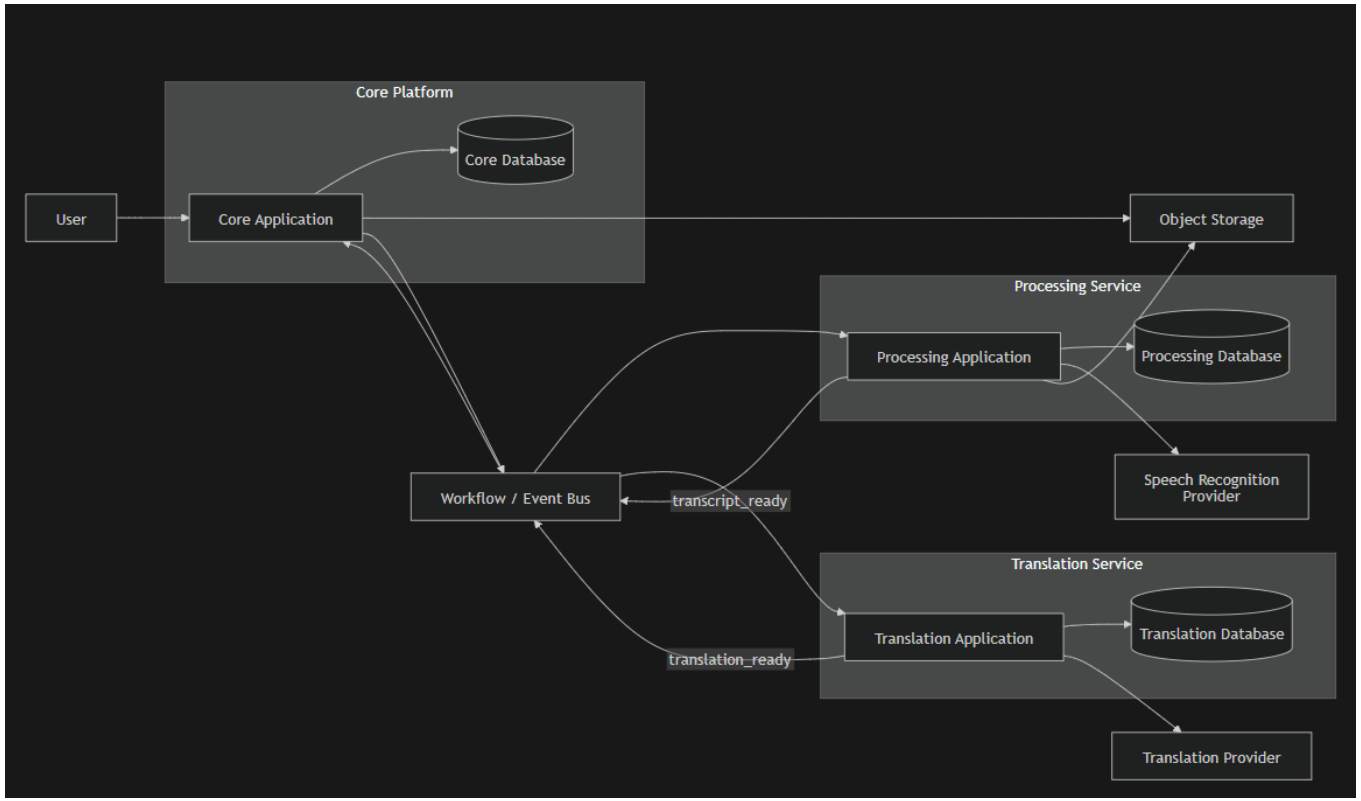
- Workflow и события становятся частью архитектуры
- Статусы синхронизируются между сервисами
- Межсервисные операции становятся идемпотентными

Этап 5. Архитектурные решения

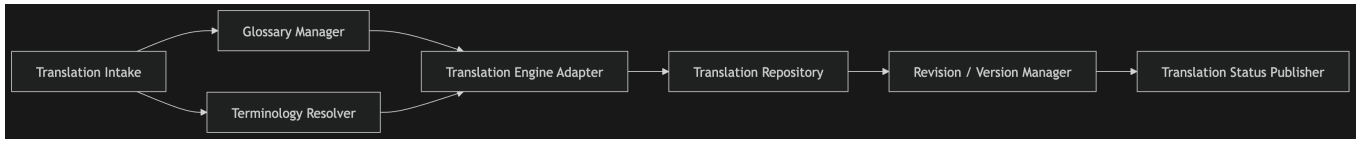
Архитектурное решение

- Перевод перестаёт быть просто одним шагом обработки видео
- Для перевода появляются свои данные и свои правила
- Поэтому перевод выделяется в отдельный сервис
- Этот сервис отвечает за тексты, словари и версии перевода

Этап 5. Перевод как отдельный сервис



Добавление отдельного Translation Service



Внутреннее устройство Translation Service

Выделение Translation

Новая граница

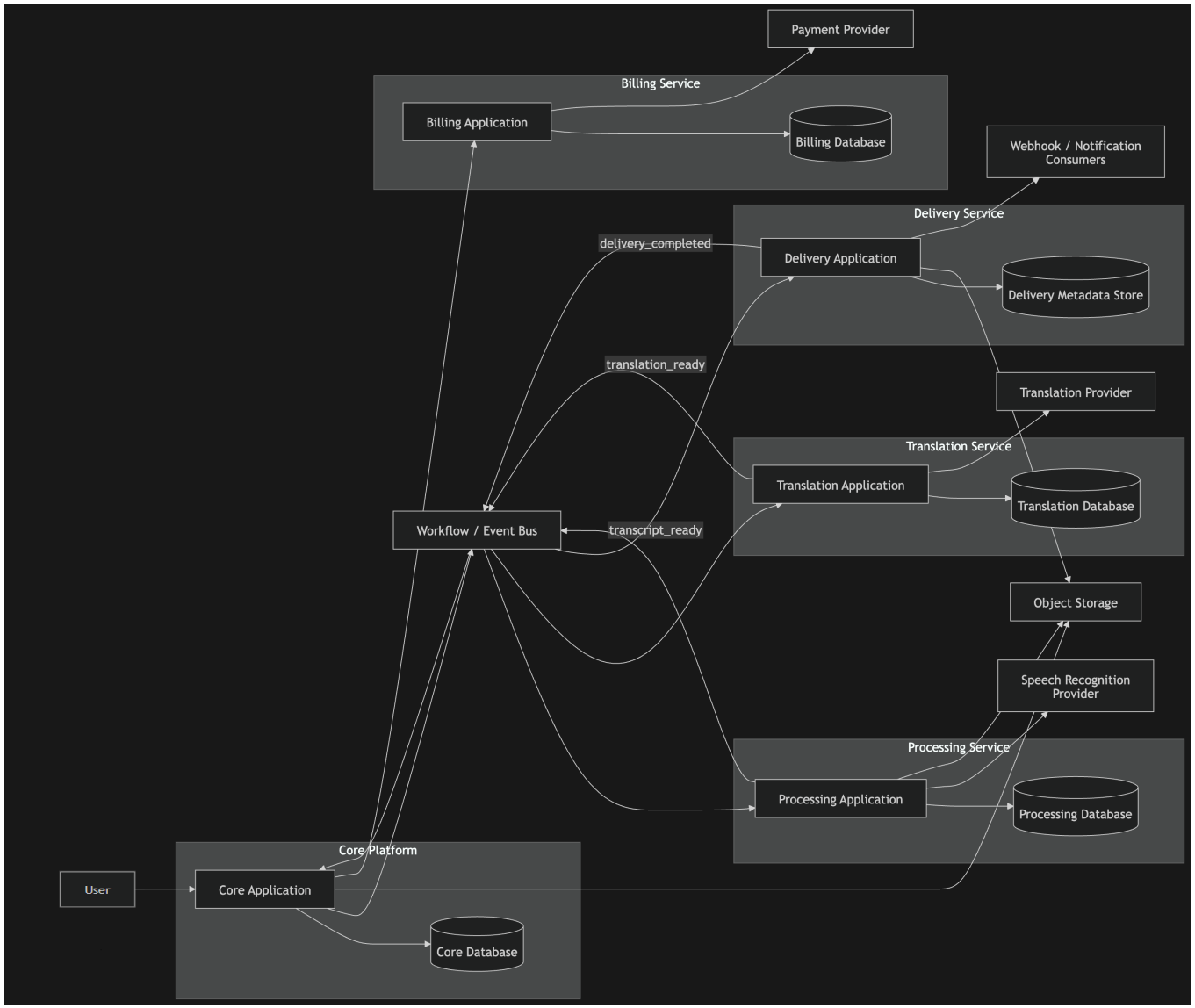
- Перевод становится самостоятельным доменом
- Вводятся словари, версии и повторное использование
- Языковая логика отделяется от processing
- Качество перевода развивается независимо

Этап 6. Архитектурные решения

Архитектурное решение

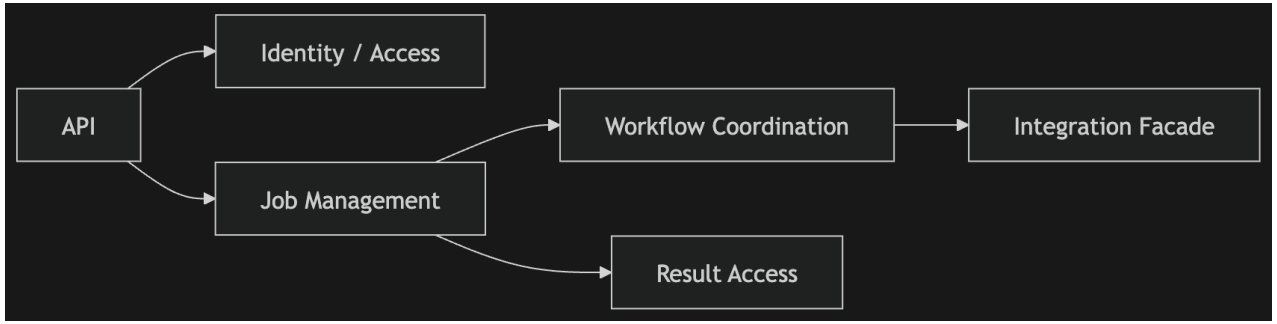
- Сформировать зрелую сервисную архитектуру вокруг устойчивых доменов
- Выделить Billing Service для квот, тарифов и списаний
- Выделить Delivery Service для сборки и публикации результата
- Сделать Core Platform координатором всей системы

Этап 6. Зрелая сервисная карта

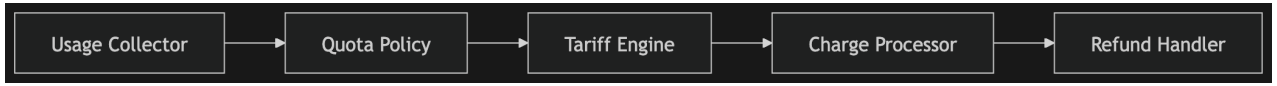


Итоговая архитектура: набор независимых сервисов

Внутренние контуры системы



Core Platform: управление системой и workflow



Billing Service: тарифы и квоты

Delivery Service: формирование и доставка результата

Зрелая сервисная архитектура

Итоговая структура

- Core Platform координирует систему
- Processing выполняет media pipeline
- Translation управляет текстом
- Billing контролирует оплату и лимиты
- Delivery отвечает за выдачу результата

Логика эволюции

Последовательность

1. Сначала фиксируется модель обработки
2. Затем строится структурированный монолит
3. Далее выделяется processing-контур
4. Потом появляются самостоятельные домены

Монолит. Практический вывод

Как смотреть на монолит

- Монолит — это не “плохая” архитектура
- Он хорошо работает, пока система и команда ещё компактны
- Его нужно проектировать модульно, а не как один большой комок логики

Микросервисы. Практический вывод

Как смотреть на микросервисы

- Это не универсально лучшая архитектура
- Они оправданы там, где действительно нужна автономия
- Хорошая граница сервиса важнее, чем сам факт дробления

Источники информации и изображений

1. https://pikabu.ru
2. https://www.architime.ru/specarch/top_10_parametric_arch/parametrism.htm
3. https://au.pinterest.com/ideas/geometric-analysis-of-gothic-structures/898619358879/
4. https://www.arsis.ru/blog/microservises-vs-monolit/

Photographs used under public domain or fair usage. No copyright infringement intended. If used accidently under wrong license please contact to be removed.

Программная инженерия

Лекция 8: Архитектура программных систем: проектирование и эволюция

Денисов Артём Владимирович

Уральский Федеральный Университет

2026